

SCORE: A Framework for Synthesizing Complete, Owned, Responsive Environments

J.P. Kardolus, Babon Innovations B.V., Utrecht, The Netherlands

Working draft, 8 October 2026

Abstract

Games, films and robot training need 3D worlds that are complete, editable and owned by the people who make them. World models and one-shot 3D generators can produce a convincing place from a sentence or a picture, but they give frames that no engine can open, or one fused scene without separate objects or clear rights. Studios have always built worlds in stages, from concept art and layout to modelling, surfacing, sound and review. We present SCORE, a world generation framework that keeps these stages and automates large parts of them with open vision foundation models, procedural tools and coding agents, while the creator chooses and reviews. Because every stage’s output is kept, the creator can rerun or fix any single stage, and because every model allows commercial use, the creator holds full commercial rights. We demonstrate SCORE on the game world of 2099 and plan three further worlds of different kinds. Open models, working in checkable stages, can build worlds that a creator owns and keeps editing. Code and paper are available at <https://github.com/Babon-Innovations-b-v/score> (MIT license).

Gap: teaser figure. This figure will show four worlds made with SCORE: the game world of 2099, a stylised underwater world, a simulation world for robots, and a third-person fantasy world. Each will appear as its chosen concept beside the finished world in its engine. It waits until the four worlds are finished and accepted by their creators.

1 Introduction

Interactive 3D worlds are the raw material of games, of film and of embodied AI, where agents are trained and tested in simulators [1]. Building such a world is still expensive. Every object has to be modelled, given surfaces, placed, given collision and physical properties, lit and given sound, all in one consistent style, and the world has to stay editable while the project changes. Generative models promise to take over much of this work, but the people who would use them are wary of it: in GDC’s 2026 survey, 36 percent of game workers used generative AI at work, and 52 percent thought it was hurting the industry [2]. Taking over the work is therefore not enough. A tool for building worlds has to keep the creator the author of the result, holding full commercial rights.

Two lines of research generate the world itself. The first, learned world models, generates it as frames. Genie learns an interactive environment from video [3], and Genie 3 renders a world that a person can move through in real time; its announcement lists a constrained range of actions and a few minutes of continuous interaction among its limits [4]. Nothing that these models produce can be opened in a game engine or an editor, and their world exists only as far as the player looks. The second line, 3D world generators, produces geometry instead. WorldGen turns a text prompt into a traversable scene for standard game engines [5], and HY-World 2.0 turns text or a picture into a single splat or mesh world [6]. Most such outputs, however, are “static monolithic assets

with limited editability and physical interaction” [7], and WorldAct and WorldSculpt address this by recovering separate objects from them after the fact [7], [8]. In both lines the world is generated in one shot. When part of the result is wrong, the creator can regenerate the whole world or repair it by hand, but cannot see which step caused the fault, and cannot change that step alone.

A third line of work keeps the world explicit. Infinigen and Infinigen Indoors build every asset from procedural rules, with materials as generators of their own [9], [10]. This gives real geometry and full control, but only over the content that the rules describe. Holodeck has a language model write layout constraints, which a solver then satisfies with assets retrieved from a library [1]. SceneCraft, recursive code world models, WorldClaw and AutoUE let coding agents write scenes, or whole games, as programs [11], [12], [13], [14]. LEGO-Anything shows what such agents need. When coding agents rebuild a scene from a single picture, they start weakly, undo their own progress while editing, and judge their own geometry unreliably; giving them grounded tools, rather than training them further, closes much of that gap [15]. All of these systems, however, either rebuild a scene that already exists or draw their objects from a fixed library of assets.

Meanwhile, each of the parts that a world needs can now be read out of pictures by an open model. Vision foundation models learned the world from images in the way that language models learned it from text [16]. They can now turn a picture into a 3D shape aligned with its pixels [17], into a mask for each named object [18], into depth in metres [19], [20], into the parts of an object [21] or into a human body [22]; related models turn a sentence into motion [23] or into sound [24]. Most of these models pass on pictures, masks, depth maps or meshes rather than their internal features, so a coding agent can connect them to one another as it would connect tools. What none of them provides is one consistent style across objects that different models have made. The field names this as an open problem [25], [26], and Hunyuan3D Studio addresses it only by restyling the input picture [27].

Studios have always built worlds in stages: concept art, layout, asset lists, modelling, surfacing, sound and review. Each stage has a clear output that a person can check before the next one starts, which makes this way of working the natural basis for a world generation framework. SCORE keeps these stages and automates large parts of them with the models above, with procedural tools and with coding agents, while the creator chooses and reviews. Its name states what it aims for. The worlds it makes are **complete**: every object, surface, light and sound exists wherever the player looks. They are **owned**: the creator decides what the world is, edits it in the tools they already use, and stays its author, holding full commercial rights, because every model in the framework allows commercial use of its output. They are **responsive**: the world answers to the physics engine that loads it, because every object has real collision, mass and material, so it behaves physically in any engine or simulator, under physics that each world may set for itself, such as the low gravity of the Moon. And they can be any kind of **environment**, real or made up. SCORE does not compete with procedural tools, part splitters or world generators; it takes them in as building blocks wherever their licences allow. It also does not generate a world in one shot: the creator chooses the look, the concept and the style of each place, and coding agents and models do the work of the stages between those choices. To our knowledge, no other openly available world generation work joins vision foundation models into engine-ready scenes that are built from reference pictures, steered by a person’s choices at each step, and made only with models that allow commercial use of their output.

This paper makes three contributions:

- **The SCORE framework:** staged, retraceable generation from a creator’s choices to one

canonical scene, which consists of a generated base layer and the creator’s own edit layers (Section 2).

- **The mechanisms that keep its worlds coherent and make faults cheap to fix:** deterministic checks before every stage that costs money, a parts check that decides how each object is built so that every real-world object becomes one model, a shared library of rule-based surfaces that keeps shape and surface apart, and review tools for the creator (Section 2).
- **Evidence from four worlds of different kinds:** the time, the cost and the faults caught before spending at every stage, and an evaluation on a shared benchmark (Sections 3 and 4).

2 Methods

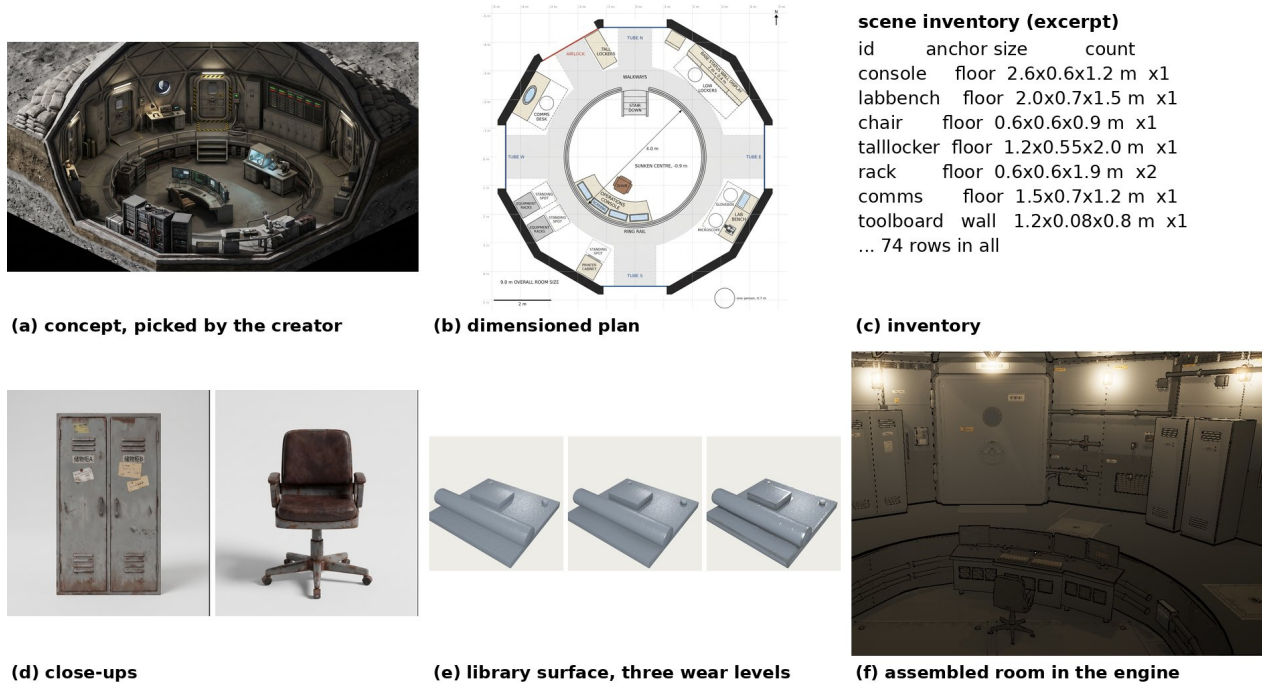


Figure 1: The stages on one place, the central hub of the game 2099. (a) The concept, drawn by a picture model from the creator’s reference pictures and chosen by the creator. (b) The dimensioned plan in metres, with walkways and standing spots. (c) An excerpt of the scene inventory, which has 74 rows. (d) Close-up pictures of two objects. (e) One surface from the shared library of rule-based surfaces at three wear settings. (f) The assembled room in the game engine.

Figure 1 shows the stages on one place. The creator chooses a concept (a), and a coding agent turns it into a dimensioned plan (b) and a scene inventory (c). A picture model draws a close-up of every object in the inventory (d). Each object is then built in code or generated in 3D, its parts are painted from the shared library of rule-based surfaces (e), and the place is assembled in an engine (f). The output of every stage is kept, so the creator can trace a weak result to the stage that caused it, rerun or fix that stage, and regenerate the stages after it while keeping their own edits. The sections below describe each stage with an example.

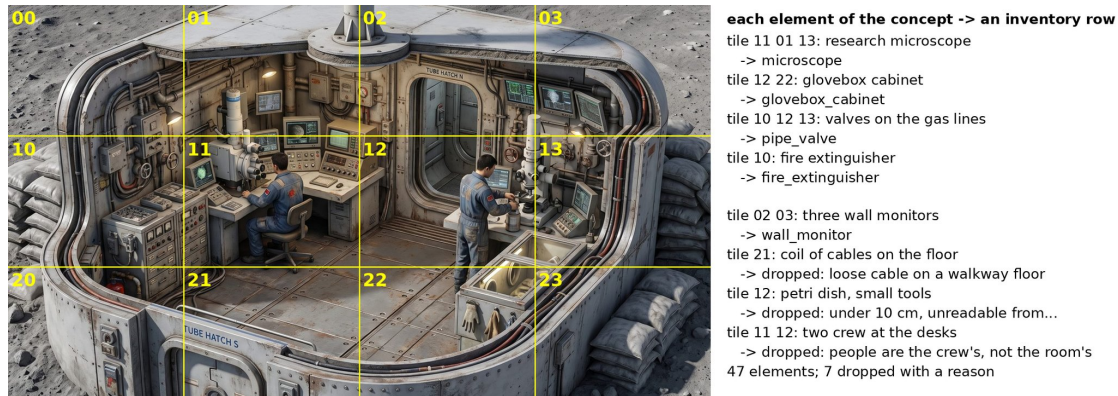


Figure 2: The inventory of the lab on the Moon base, built from its concept. Left: the concept, cut into twelve tiles. Right: examples of how each visible element maps to an inventory row, or is dropped with a written reason. Of the 47 elements in this concept, 40 were mapped to inventory rows and 7 were dropped with a written reason.

2.1 Inventory as a partition of the concept

The inventory decides what exists in a place. A coding agent cuts the concept into tiles and lists every visible element, and each element either becomes a row (with its kind, anchor, size and count) or is dropped with a written reason (Figure 2). Nothing is made or placed that is not a row. After the build, the place is rendered from the concept’s own camera and compared with the concept; a place that is clearly sparser than its concept fails this concept-density check.

2.2 Code or generation: the parts check

Picture-to-3D models build chunky solids well, but flat panels, thin beams and openings badly, while code builders are exact and straight but show only the parts that the coding agent wrote. For each kind of object, the parts check renders a code build from the front and compares it with the close-up picture: if every part of the close-up is visible in the build, the kind is built in code, and otherwise it is generated from the close-up with Pixa3D [17] and made solid (Figure 3). Composite objects are split in the inventory, so that every real-world object becomes one model: a console, for example, becomes a desk carrying separate monitors and keyboards.

2.3 Shape and surface apart

Models and code builders give shape only. Every part of every object takes one surface from a shared library of rule-based surfaces, written as ProcFunc functions [28] that are built on Infinigen’s shaders [9] and take their colours from the place’s palette (Figure 4a). A room has one wear setting, and wear is applied where it has a cause, such as edges and the places where feet pass. On a generated model, PartCrafter [21] splits the shape into parts, and the model’s picture only chooses which library surface each part takes. Figure 4b shows why: when each piece kept the colours of its own picture, every piece brought its own rust and stains, and the room looked inconsistent. Sound is attached in the same way, to surfaces, rooms and objects, and is generated by MOSS-SoundEffect [24] (Appendix F).

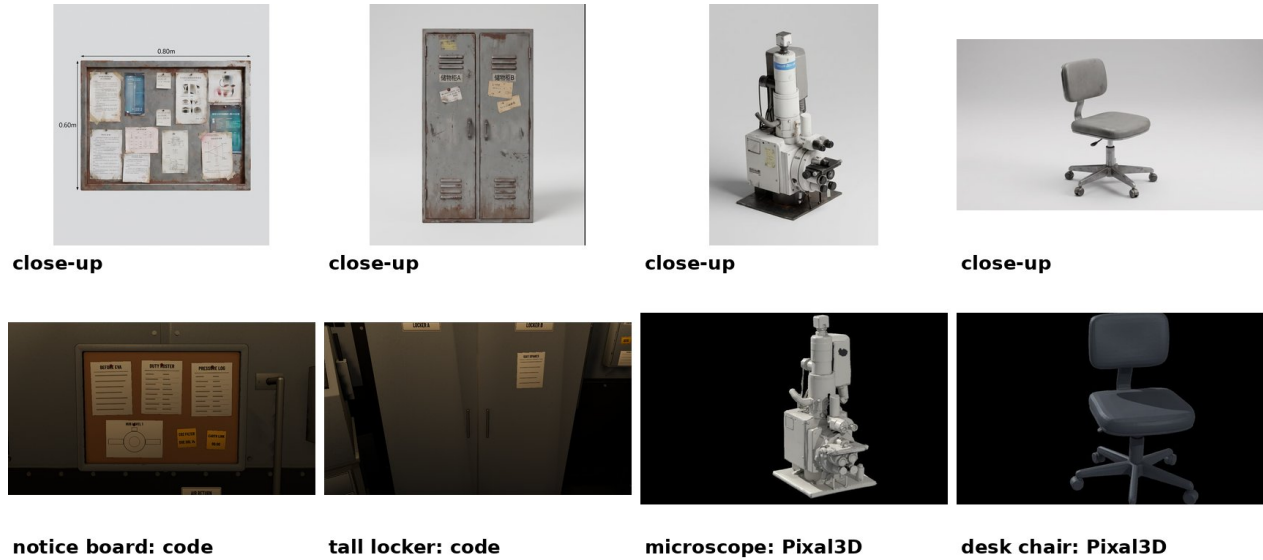


Figure 3: The parts check decides how each object is built. Top: the close-up picture of four objects. Bottom: the object as built. The notice board and the tall locker are built in code, because their code builds show every part of their close-ups. The microscope and the desk chair are generated with Pixal3D, because no code build shows all of their parts.

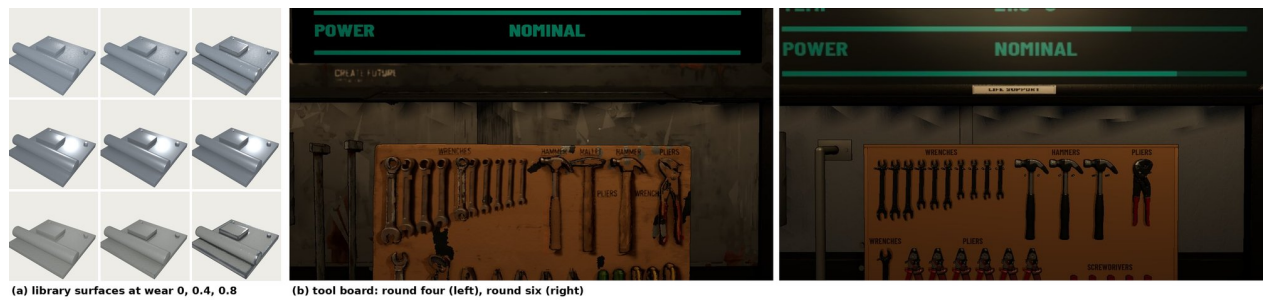


Figure 4: (a) Three surfaces from the shared library of rule-based surfaces (painted panel, bare steel and deck plate) at wear settings 0, 0.4 and 0.8. (b) The hub's tool board when each generated piece kept the colours of its own picture (left), and after every surface came from the library and each tool became its own model (right). Both shots use the same camera and the same build of the game.

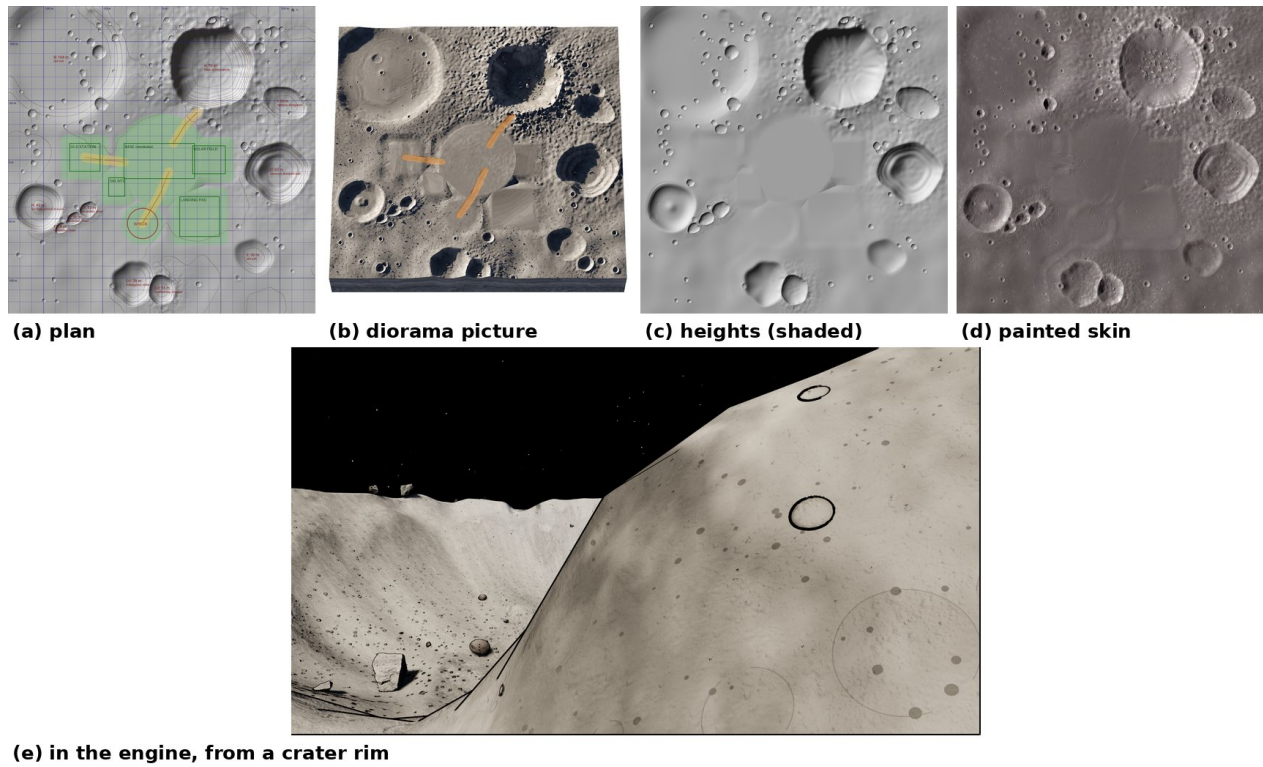


Figure 5: The stages for large outdoor places, on the ground around the Moon base. (a) The dimensioned plan with the base's flat areas and paths. (b) A diorama picture drawn over the plan. (c) The heights that follow the diorama, shown shaded. (d) The skin painted over the plan. (e) The result in the engine, seen from a crater rim.

2.4 Terrain

Large outdoor places follow the same pattern (Figure 5). A dimensioned plan places the flat areas and paths the place needs (a), a diorama picture drawn over the plan sets the landforms (b), and the heights follow the picture (c). A skin is painted over the plan, with fine relief from monocular depth [20] (d), rocks are single generated models, and everything is built on the world’s own curved ground (e).

2.5 Checks before spending



Figure 6: A fault caught by the placement check. (a) In the first build of the hub, a conduit box covered the porthole. (b) In the current framework, the placement check found the box in front of the opening and moved it 0.5 m along the wall before anything was spent.

Deterministic checks run before every stage that costs money. They test real geometry rather than relying on an agent’s judgement, because a fault costs nothing to fix in the plan but an evening of the creator’s time once the place is built. The checks look for light leaking out of the room, pieces in front of openings or off their surface, doors that do not separate their two sides, models that are open, too thin or leaning, and surfaces off the palette (Appendix D). Figure 6 shows one fault the placement check caught. A failed check sends the work back one stage.

2.6 The canonical scene

Every place is exported as one OpenUSD stage [29] in which each object carries its kind, its transform in metres, its collision, the library surface of each part and that surface’s sounds, with the generated base layer under an edit layer that belongs to the creator, so that an edit survives when the base is generated again (Figure 7).

Gap: the scene in a game engine. The same stage loaded in a game engine, with its collision and surfaces, and the mass and friction of each object written into it. It waits on the game engine adapter and on the framework recording mass and friction.

2.7 Review tools

Each place ends with the creator’s review. The review tools that SCORE supplies are pages that show each stage’s output side by side, before-and-after shots from the same cameras, walkthroughs inside the engine, and the result of every check. An optional world step can turn the concept into



(a) the game (Godot): glTF models and a JSON layout



(b) the OpenUSD stage in Blender, same camera



(c) Blender's outline over the game's picture



(d) an edit layer over a regenerated base

Figure 7: The wreck, one outdoor place of the game 2099, as an OpenUSD stage. (a) The place in the game, from one of its fixed cameras. (b) The stage loaded in Blender and rendered from the same camera. (c) Blender's object outlines drawn over the game's picture. (d) The same camera after an edit in the edit layer, made here by a script, that moved the round cover and gave the capsule's painted hull another library surface, and after the base layer was generated again.

a whole room that can be walked through, so that walls and objects the concept does not show can be seen; nothing it produces is shipped.

For the wreck, the review page is a static folder built only from the files the stages wrote, with the assembled scene rendered by Blender from the place’s OpenUSD stage from fixed cameras and along a walk round it (Figure 8).

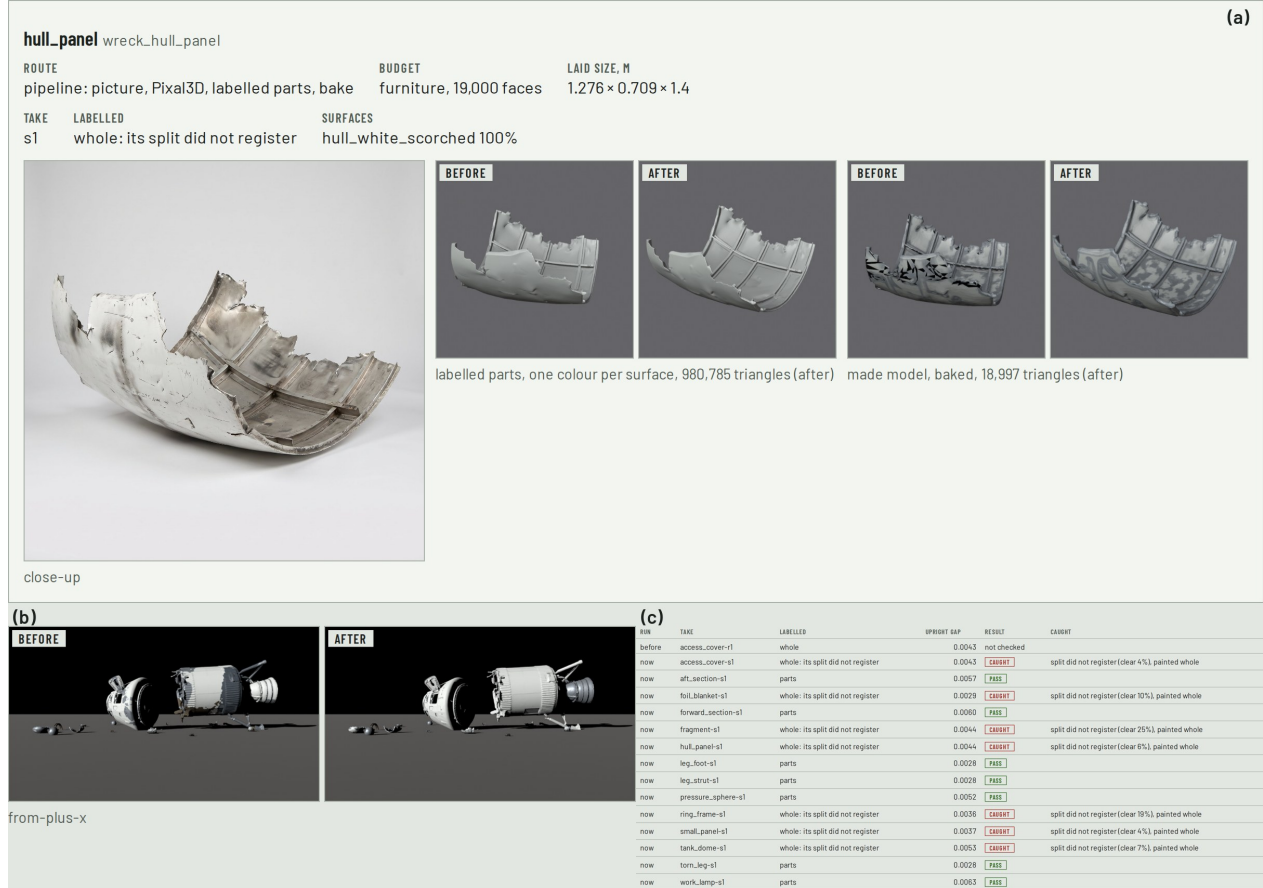


Figure 8: The wreck’s review page, from two runs of the route over the same place. (a) One model: its close-up, its labelled parts (one colour per library surface) and its baked model, each from the first run and the rerun, drawn from the same camera. (b) The assembled scene from one fixed camera, before and after the rerun. (c) The parts check for each take, with what it caught in the rerun: the split of seven takes did not register against their pictures, so each was painted whole.

3 Results across worlds

We build four worlds with the same stages: the game world of 2099 (a Moon base and its surroundings, a camp on Mars, and a prologue on Earth), a stylised underwater world, a simulation world for robots, and a third-person fantasy world.

Gap: Table 1, results per world and stage. For each world, this table will give the number of places, the wall-clock time, the cost in GPU time and in picture calls, the number of models built in code and generated, and the faults caught before spending at each stage against those found afterwards. It waits until world 1 is finished and reviewed by its creator (most of its places are built, but none after its first room has yet been reviewed in play) and until worlds 2 to 4 are built. The records for each place will be in Appendix B.

Gap: figure of one place per world. This figure will show one representative place from each world beside its concept, rendered from the concept’s own camera. It waits on the same worlds; every place will be shown in Appendix A.

4 Evaluation

Gap: LEGO-Bench. This section will report SCORE’s scores on LEGO-Bench [15], a benchmark of scenes rebuilt from pictures. It waits until the benchmark’s code is released and until SCORE’s scene export and its Blender adapter are built.

Gap: ablations. This section will compare the framework with and without the optional world step, and with the checks before spending switched off, on the same places. It will measure how much of the concept is covered, how dense the built place is, the cost, the faults found after spending, and the creator’s review. It waits until every stage runs from the framework repository.

Gap: style coherence. This section will measure whether a place keeps one style, using rendered frames: the colour distance to the palette for each kind of source, the spread of lightness, and the density of ink lines, validated against the creator’s reviews. The measure is designed but not yet built.

Gap: robots in the simulation world. This section will show a rigged robot loaded together with the simulation world in a robotics simulator, with masses, friction and joints written into the scene, and with motion from the framework’s motion models. It waits on world 3.

5 Limitations

- **One creator, one world so far.** Every verdict on the results is one creator’s, and only the first world has been built.
- **Code builders do not scale well.** They give clean objects, but a coding agent has to write one for every kind of object.
- **Places come out sparser than their concepts.** The concept-density check catches this but does not fix it.
- **One stage depends on a closed, paid picture model.** Nano Banana Pro, a closed and paid service, draws the concepts and close-ups; its daily limits set the pace of world 1, and it can change or be retired, which limits reproducibility and leaves a public framework depending on a closed service for one stage. A cheaper model of the same family did not replace it (Appendix E).
- **Objects are physical but not interactive.** Seats, terminals and other usable objects do not work in the output.

6 Future work

- Build the three further worlds: a stylised underwater world, a simulation world for robots, and a third-person fantasy world.

- Add a game engine adapter for the OpenUSD scene, and write each object’s mass and friction into it.
- Make assets interactable, which robotics in particular needs.
- Replace the closed picture model for close-ups: an open picture model, run in our own cloud batches, is being tested for this stage.
- Wrap open solvers as stages for worlds at other scales, starting with orbital dynamics through REBOUND [30].
- Evaluate SCORE on LEGO-Bench [15].

7 Conclusion

SCORE builds complete, owned and responsive worlds from a creator’s choices, by joining open vision foundation models, procedural tools and coding agents into fixed stages whose outputs are kept. On its first world, the parts check, one model per real-world object and the shared library of rule-based surfaces made a room consistent, and the deterministic checks before every stage that costs money caught faults before money was spent. Whether the framework holds across different kinds of worlds is what the remaining worlds and the benchmark will show.

References

- [1] Y. Yang *et al.*, “Holodeck: Language guided generation of 3D embodied AI environments,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.
- [2] D. Argüello, “36 percent of game workers use generative AI, but half think it’s bad for the industry.” *Game Developer*, Feb. 2026.
- [3] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, *et al.*, “Genie: Generative interactive environments,” *arXiv preprint arXiv:2402.15391*, 2024.
- [4] J. Parker-Holder and S. Fruchter, “Genie 3: A new frontier for world models.” *Google DeepMind blog*, Aug. 2025. Available: <https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/>
- [5] D. Wang, H. Jung, T. Monnier, K. Sohn, *et al.*, “WorldGen: From text to traversable and interactive 3D worlds,” *arXiv preprint arXiv:2511.16825*, 2025.
- [6] Tencent Hunyuan, “HY-World 2.0.” *GitHub repository*, 2026. Available: <https://github.com/Tencent-Hunyuan/HY-World-2.0>
- [7] J. Hu, J. Guo, J. Cen, C. Yang, S. Li, and W. Shen, “WorldAct: Activating monolithic 3D worlds into interactive-ready object-centric scenes,” *arXiv preprint arXiv:2605.15843*, 2026.
- [8] M. Niu, J. He, R. Yu, L. Fu, *et al.*, “WorldSculpt: Generating compositional worlds from grounded videos,” *arXiv preprint arXiv:2609.05416*, 2026.
- [9] A. Raistrick *et al.*, “Infinite photorealistic worlds using procedural generation,” *arXiv preprint arXiv:2306.09310*, 2023.
- [10] A. Raistrick *et al.*, “Infinigen Indoors: Photorealistic indoor scenes using procedural generation,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.
- [11] Z. Hu, A. Iscen, A. Jain, T. Kipf, *et al.*, “SceneCraft: An LLM agent for synthesizing 3D scene as Blender code,” in *International conference on machine learning (ICML)*, 2024.

- [12] Z. Li, Y. Liao, and B. Zhu, “Recursive code world models: Building complex worlds through recursive scene programs,” *arXiv preprint arXiv:2609.11499*, 2026.
- [13] C. Guo, J. Li, Y. Li, and Z. Huang, “WorldClaw: Agentic 3D open-world generation at scale,” *arXiv preprint arXiv:2608.05248*, 2026.
- [14] L. Yin, W. Cheng, Z. Qin, T. Huang, *et al.*, “AutoUE: Automated generation of 3D games in Unreal Engine via multi-agent systems,” *arXiv preprint arXiv:2603.07106*, 2026.
- [15] X. Li *et al.*, “LEGO-Anything: Coding agents for 3D scene reconstruction,” *arXiv preprint arXiv:2609.36380*, 2026.
- [16] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, “DINOv2: Learning robust visual features without supervision,” *arXiv preprint arXiv:2304.07193*, 2023.
- [17] D.-Y. Li *et al.*, “Pixal3D: Pixel-aligned 3D generation from images,” *arXiv preprint arXiv:2605.10922*, 2026.
- [18] N. Carion *et al.*, “SAM 3: Segment Anything with concepts,” *arXiv preprint arXiv:2511.16719*, 2025.
- [19] R. Wang *et al.*, “MoGe-2: Accurate monocular geometry with metric scale and sharp details,” *arXiv preprint arXiv:2507.02546*, 2025.
- [20] L. Yang, B. Kang, Z. Huang, *et al.*, “Depth Anything V2,” *arXiv preprint arXiv:2406.09414*, 2024.
- [21] Y. Lin *et al.*, “PartCrafter: Structured 3D mesh generation via compositional latent diffusion transformers,” in *Advances in neural information processing systems (NeurIPS)*, 2025.
- [22] X. Yang, D. Kukreja, D. Pinkus, others, and K. Kitani, “SAM 3D Body: Robust full-body human mesh recovery,” *arXiv preprint arXiv:2602.15989*, 2026.
- [23] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, *et al.*, “Kimodo: Scaling controllable human motion generation,” *arXiv preprint arXiv:2603.15546*, 2026.
- [24] OpenMOSS-Team, “MOSS-SoundEffect v2.0.” Hugging Face, 2026. Available: <https://huggingface.co/OpenMOSS-Team/MOSS-SoundEffect-v2.0>
- [25] J. Wu, Z. Lou, J. Liu, D. Du, C. Guo, and S. Guo, “From visual synthesis to interactive worlds: Toward production-ready 3D asset generation,” *arXiv preprint arXiv:2604.23629*, 2026.
- [26] Z. Yang *et al.*, “FlowScene: Style-consistent indoor scene generation with multimodal graph rectified flow,” *arXiv preprint arXiv:2603.19598*, 2026.
- [27] B. Lei, Y. Li, X. Liu, *et al.*, “Hunyuan3D Studio: End-to-end AI pipeline for game-ready 3D asset generation,” *arXiv preprint arXiv:2509.12815*, 2025.
- [28] A. Raistrick *et al.*, “ProcFunc: Function-oriented abstractions for procedural 3D generation in Python,” *arXiv preprint arXiv:2604.26943*, 2026.
- [29] Pixar Animation Studios and Alliance for OpenUSD, “OpenUSD: Universal scene description.” 2026. Available: <https://openusd.org/>
- [30] H. Rein and S.-F. Liu, “REBOUND: An open-source multi-purpose N-body code for collisional dynamics,” *Astronomy & Astrophysics*, vol. 537, p. A128, 2012.
- [31] Google, “Gemini 3 Pro Image (‘Nano Banana Pro’) through the Gemini API.” 2026. Available: <https://ai.google.dev/>
- [32] O. Siméoni, H. V. Vo, M. Seitzer, *et al.*, “DINOv3,” *arXiv preprint arXiv:2508.10104*, 2025.
- [33] World Labs, “Terms of Service.” 2026. Available: <https://docs.worldlabs.ai/terms-of-service>
- [34] Black Forest Labs, “FLUX.2 [klein] 4B model card.” Hugging Face, 2026. Available: <https://huggingface.co/black-forest-labs/FLUX.2-klein-4B>
- [35] Anthropic, “Claude Code.” 2026. Available: <https://www.anthropic.com/claude-code>

Appendix

A Per-world demos

Gap. This appendix will show every place of every world as its concept beside the finished build, with links to demo videos and playable builds. It waits until the worlds are finished.

B Detailed per-stage results

Gap. For every place, this appendix will give the wall-clock time, the cost, the number of models built in code and generated, and every fault caught before or found after spending, by stage and by check. Records already exist for 21 place runs of world 1, but they are not results until that world is finished and reviewed; worlds 2 to 4 have not been started.

C Development history

The framework grew out of building the game 2099 between late September and early October 2026, with one owner directing coding agents. In its first form it joined four parts: rooms laid out in code, a scene plan from a world model that painted depth panoramas of those rooms, props from a picture-to-3D pipeline, and coding agents that fitted each room to its plan. Eight places were fitted out to their plans in a single day. The owner found that their layouts were close to the plans, but that their styles were mixed, with models from earlier rounds and with objects that the plans never showed. The cause lay in the workflow: every pass added things to a room, and no pass checked what had been placed against the plan. This led to the inventory as a strict list, with nothing made or placed outside it, and to deciding the look before any money is spent. The structure of each room then came from concept cutaways drawn from the owner’s own reference pictures; for the hub he picked one cutaway out of twenty. Scale moved into a dimensioned plan drawn before the concept, after the first cutaway came out at doll’s-house scale: about 13 m across, judged by the people drawn in it, against the 8 to 9 m that had been asked for.

C.1 The hub in six rounds

The mechanisms of Section 2 then came out of six rounds of work on one room, the central hub of the game 2099, on 6 and 7 October 2026. Each round answered the creator’s verdict on the round before (Table C1). In the first two rounds, the creator chose between the old and the new method by comparing them side by side from the same cameras.

Table C1. The hub’s six rounds. Cost is given as rented GPU time in euros and picture-model calls in dollars. Frame time is the mean frame time on an RTX 5080, in the night scene, at 1600×900.

Round	Change	Creator’s verdict	Cost	Frame time
1, two walls, A/B	surface library, routing by shape, room checks	chose the new method (“Y looks much cleaner”)	€0.88	7.81 ms vs 7.53
2, two walls, A/B	71-variant library, shared texture sets, roof check	chose the new method again	€1.70	8.05 ms vs 8.24

Round	Change	Creator’s verdict	Cost	Frame time
3, whole room	all stages end to end, 74 models	old models were still loaded; labels and floor fittings were wrong	€1.67	6.78 ms (old 9.22)
4, whole room	detail through the pipeline, picture colours kept	“pretty much all of this became messy”	€8.50, \$5.09	9.6 ms
5, eight pieces, A/B	shape only from the pipeline, library surfaces, parts check	new method preferred for the door, notice board and porthole; three pieces still wrong	€0.21	not run
6, whole room	round 5 everywhere, one model per object, straightness check	“this looks amazing, I have no negative feedback”	€3.48, \$1.07	5.6 ms

Three findings shaped the method. First, deterministic checks catch what agents miss. In round one, the share of rays that leaked out through walls and roof fell from 1.53 percent of 5,817 rays to zero, and the colour noise within surfaces fell from a median of 31.1 to 3.2 percent. Only walking through the loaded room, however, showed that 26 old models were still in it, and that finding became the made-only check. Second, when each generated piece kept the colours of its own picture, every piece brought its own wear, and the room looked inconsistent; surfaces have since come only from the library. Third, generated composite objects melted their parts together, and generated boxes leaned (one locker by 13 degrees). This led to one model per real-world object and to the straightness check (Figure 4b). By round six the room used 0.45 million triangles, against 1.96 million in round four, and 187 MB on disk, against 487 MB.

D Check details

- **Room checks** take about two seconds per room and run on the laid-out meshes. They cast rays to find light leaking out through walls and roof, find pieces placed in front of a real opening, find roof and wall fittings more than 3 cm off their surface or turned more than 5 degrees from it, and count pieces, triangles and lamps against the room’s budget.
- **Door check:** every door must stand in a wall or partition that fully separates its two sides. The check tests this by walking a grid from one side to the other.
- **Concept-density check:** the concept picture is cut into crops, and every visible element must have an inventory row or a written reason for leaving it out. After the build, the place is rendered from the concept’s camera and shown beside the concept.
- **Parts check:** in the code build, every part shown in the close-up picture must be at least 10 percent visible from the front, and no label may cover a part.

- **Model check:** every model must be a closed solid with walls at least 3 mm thick. A generated box fails if one of its sides tilts more than 2 degrees or is warped. The step that makes a generated model solid stops itself if the model’s bounding box grows more than 3 percent.
- **Palette sweep and storage budget:** the colours of every model are compared with the place’s library palette, and each room may use at most 250 MB on disk and 640 MB of textures on the graphics card.
- **Scene check and made-only check:** in the engine, these checks find anything that floats, sinks or overlaps, and any visible mesh that the framework did not make. A made-only finding can never be waived.

E Models and licences

Every model in the framework must allow commercial use of its output (Table E1). Candidate models were dropped under this rule; among them were NVIDIA’s Lyra 2.0, whose weights are licensed for internal research only, and Hunyuan3D 2.1, whose licence does not apply in the European Union. We read each licence for its clauses on commercial use. Four of the models read pictures through Meta’s DINO family of models, each through its own copy. What passes from one model to the next is always pictures, masks, depth maps or meshes, never one model’s internal features.

Table E1. Models in the framework.

Model	Role	Licence
Nano Banana Pro [31]	concepts, close-ups, ground skins	paid API
Pixal3D [17]	picture to 3D	MIT; DINOv3 licence [32] for its encoder
PartCrafter [21]	parts	MIT; its non-commercial background remover not used
Depth Anything V2 Small [20]	ground relief	Apache-2.0
ProcFunc, Infinigen shaders	surfaces	BSD-3-Clause
MOSS-SoundEffect v2.0, CLAP	sound	Apache-2.0
World Labs Marble [33]	optional world step	paid; outputs owned by paid users
FLUX.2 klein 4B [34]	earlier prop pictures	Apache-2.0
MoGe-2 [19], SAM 3 [18]	measuring and finding objects in pictures	MIT; SAM Licence
Kimodo [23], SAM 3D Body [22]	motion from sentences; body shape from a picture	Apache-2.0 code, NVIDIA Open Model License; SAM Licence

On our tier, Nano Banana Pro allows 250 pictures a day and 20 a minute; the cheaper Nano Banana (gemini-2.5-flash-image) allows 2,000 a day and 500 a minute. We tested whether the cheaper model could draw the close-ups instead, on the same 10 lab objects with the same prompts and inputs (Table E2). It refused the standard prompt for 6 of the 10 objects and drew those only from the crop alone, and its pictures were about one megapixel with the object small in the frame. Close-ups therefore stay on Nano Banana Pro.

Table E2. Ten lab close-ups drawn by Nano Banana instead of Nano Banana Pro.

Result	Objects
good	2
usable	3
marginal	1
wrong shape (microscope, chair, sample tray)	3
refused (glovebox)	1

The picture-to-3D step, the part splitting, the surface bakes, the sound generation, all checks and the assembly run automatically. Coding agents (Claude Code [35]) write the plans, the inventories and every code builder; in the hub’s final round, 86 of its 98 models were built in code. Some steps were still done by hand: some generated objects were turned to face the right way by eye, and a few labels were placed by hand before a rule took over that job. The concepts and the verdicts are the creator’s own choices.

F Surfaces, sound and light

The surface library holds families of variants: painted, steel, aluminium, rubber, plastic, cable, fabric, composite, glass, screen, light and print. After the hub’s second round it held 71 variants made from 14 ProcFunc recipes. Each variant is a recipe with settings, takes its colours only from the palette, and is baked into base-colour, metal-roughness and normal maps at a texture density that depends on how close the player can get. Wear has three levels and comes from causes: edges, and kicks within 32 cm of the floor that a piece stands on. Every sound starts from a short written brief about its object. Four takes are generated for each sound, scored by how well CLAP matches them to their prompt minus any measured faults (clipping, unsteady loops or bad joins), and levelled to one loudness per kind of sound under a true peak of -3 dBTP. A creator may swap any take. For the game 2099, all 61 sounds were generated as 244 takes in one batch for €1.61, and the creator chose to keep earlier recordings for three kinds of sound: breathing, a sliding door, and footsteps on steel.

Light adds up, so it is planned as layers. For each kind of module, the bounce light of each group of lamps, and of a few sun directions through the windows, will be baked on rented machines as a separate layer, and the engine will mix these layers live according to each lamp’s brightness. Outdoor light and the light on moving objects stay live. This part is not built yet.

G Cost and infrastructure

Every heavy step runs on rented machines that are deleted after each batch: NVIDIA graphics cards for picture-to-3D, part splitting and baking, and processor-only machines when no cards are in stock. The engine’s tests, its screenshots and its frame-time benchmark also run on rented machines. A full test run took 542 s there, against 596 s on the local computer, and frame times measured on an L4 card are converted to the local RTX 5080 by a measured factor of 3.47. A picture costs about \$0.134 at the resolution we use. The picture model’s limits on speed are described under Limitations.

A batch asks for a class of machine, such as one card with 24 GB, and takes whatever the provider has in stock across three zones and three card types, because on 8 October the cheapest type was out of stock in one zone for most of the day. Table G1 compares the same jobs on two card types. An H100 runs ten Pixa3D takes at once in 31 GB of its 80 GB, and makes nearly three times as many models an hour as an L4. It has no ray-tracing cores, so it bakes 2.6 times slower than an L4. Pixa3D therefore takes 80 GB cards first, and a bake takes an H100 only when no other card has been free for five minutes. The L40S was out of stock on both attempts, and the provider’s P100 machines started but their driver did not see the card.

Table G1. The same jobs on two card types, 8 October 2026. Wait is the time from the order until the machine answers; machine time is the whole rental, setup included.

Card	Job	At once	Wait	Machine time	Cost	Per take
L4, 24 GB	Pixa3D, 3 close-ups	3	0.4 min	28.0 min	€0.38	€0.13, 6.4 an hour
H100, 80 GB	Pixa3D, the same 3	3	0.6 min	13.9 min	€0.67	€0.22, 12.9 an hour
H100, 80 GB	Pixa3D, 10 close-ups	10	0.6 min	33.1 min	€1.62	€0.16, 18.1 an hour
L4, 24 GB	bake: 60 surfaces at 3 wear levels	1	0.4 min	3.8 min (bake 138 s)	€0.05	
H100, 80 GB	the same bake	1	0.6 min	7.5 min (bake 360 s)	€0.38	

Gap: cost tables. These tables will give the cost per kind of batch and per world, reconciled with the cloud provider’s bill. They wait on the worlds; the figures per place so far are lower bounds taken from the build logs.